



---

UNIVERSIDAD

Nicolás Felipe Bernal Gallo

Juan Daniel Bogotá Fuentes

Desarrollo Orientada a objetos  
DOPO LAB

Laboratorio #5 Interfaz  
15/11/2025

PROFESOR: María Irma Díaz Rozo

## PROGRAMACIÓN ORIENTADA A OBJETOS

### INTERFAZ

2025-2

Laboratorio 5/6

#### OBJETIVOS

1. Desarrollar una mini-aplicación gráfica considerando el patrón MVC.
2. Implementar el esquema de manejo de eventos con clases anónimas
3. Experimentar el comportamiento de las ventanas JFrame, JDialog y JOptionPane
4. Seleccionar los lienzos más apropiados para un diseño: JPanel y JTabbedPane
5. Revisar las posibilidades de los estilos:FlowLayout, BorderLayout y GridLayout
6. Apropiar algunos componentes básicos: JLabel, JTextField, JButton, JMenuBar,
7. Apropiar algunos componentes especiales: JFileChooser y JColorChooser
8. Vivenciar las prácticas XP: [Acceptance tests](#) *are run often and the score is published when a bug is found* *tests are create.*

#### ENTREGA

1. Incluyan en un archivo .zip los archivos correspondientes al laboratorio. El nombre debe ser los dos apellidos de los miembros del equipo ordenados alfabéticamente.
2. Deben publicar el avance al final de la sesión y la versión definitiva en la fecha indicada en los espacios correspondientes.

#### CONTEXTO

El objetivo es implementar el juego **SlowTetris**

El trabajo se debe hacer desde **CONSOLA**

|                                           |                                          |
|-------------------------------------------|------------------------------------------|
| El propuesto por ustedes<br>SlowTetrisGUI | El acordado en laboratorio<br>SlowTetris |
| <b>Vista - Controlador</b>                | <b>Modelo</b>                            |

**Para la capa de presentación NO deben hacer pruebas de unidad ni diagramas de secuencia**

## DESARROLLO

### Directorios

El objetivo de este punto es construir un primer esquema para el juego **SlowTetris**

1. Preparen un directorio llamado **SlowTetris** con los directorios src y bin y los subdirectorios para presentación, dominio y pruebas de unidad. Capturen una pantalla con la estructura.

- Abrimos la consola con win + r
- Nos ubicamos en escritorio
- Utilizamos el comando "mkdir SlowTetris && cd SlowTetris && mkdir src bin docs && cd src && mkdir domain presentation test"

```
C:\Users\juan.bogota-f>cd downloads
C:\Users\juan.bogota-f\Downloads>mkdir SlowTetris
C:\Users\juan.bogota-f\Downloads>cd SlowTetris
C:\Users\juan.bogota-f\Downloads\SlowTetris>mkdir src bin
C:\Users\juan.bogota-f\Downloads\SlowTetris>cd src
C:\Users\juan.bogota-f\Downloads\SlowTetris\src>mkdir presentation domain test
C:\Users\juan.bogota-f\Downloads\SlowTetris\src>cd ..
C:\Users\juan.bogota-f\Downloads\SlowTetris>tree
Folder PATH listing
Volume serial number is 9437-6886
C:.
├── bin
├── src
│   ├── domain
│   ├── presentation
│   └── test
```

- Luego estando en la carpeta de escritorio utilizamos el comando "tree" para obtener el esquema.

```
C:\Users\juan.bogota-f\Downloads\SlowTetris>cd bin
C:\Users\juan.bogota-f\Downloads\SlowTetris\bin>mkdir presentation domain test
C:\Users\juan.bogota-f\Downloads\SlowTetris\bin>cd ..
C:\Users\juan.bogota-f\Downloads\SlowTetris>tree
Folder PATH listing
Volume serial number is 9437-6886
C:.
├── bin
│   ├── domain
│   ├── presentation
│   └── test
├── src
│   ├── domain
│   ├── presentation
│   └── test
```

### Ciclo 0: Ventana vacía – Salir [En\*.java y lab05.doc]

El objetivo es implementar la ventana principal de **SlowTetris** con un final adecuado desde el icono de cerrar. Utilizar el esquema de [prepareElements-prepareActions](#).

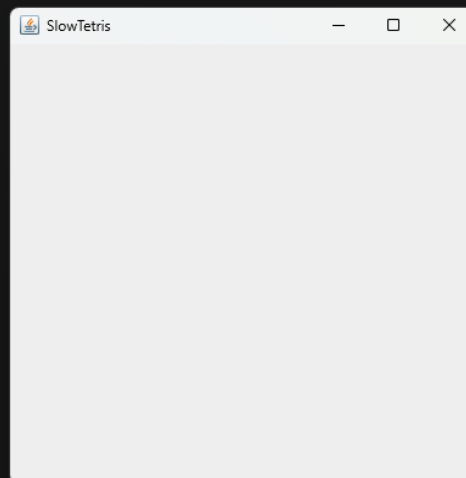
1. Construyan el primer esquema de la ventana de **SlowTetris** únicamente con el título “**SlowTetris**”. Para esto cree la clase **SlowTetrisGUI** como un **JFrame** con su creador (que sólo coloca el título) y el método **main** que crea un objeto **SlowTetrisGUI** y lo hace visible. Ejecútenlo. Capturen la pantalla.  
(Si la ventana principal no es la inicial en su diseño, después deberán mover el main al componente visual correspondiente)

```
1  import java.awt.*;
2  import java.awt.event.*;
3  import javax.swing.*;
4  import javax.swing.event.*;
5  import java.util.*;
6
7  public class SlowTetrisGUI extends JFrame{
8
9
10     public SlowTetrisGUI(){
11         setTitle(title: "SlowTetris");
12         setSize(width: 400, height: 400);
13     }
14
15     Run | Debug
16     public static void main(String[] args){
17         SlowTetrisGUI gui = new SlowTetrisGUI();
18         gui.setVisible(b: true);
19         gui.setLocationRelativeTo(c: null);
20     }
21 }
```

```
PS C:\Users\juan.bogota-f\Downloads\SlowTetris> javac -d bin src/presentation/*.java
```

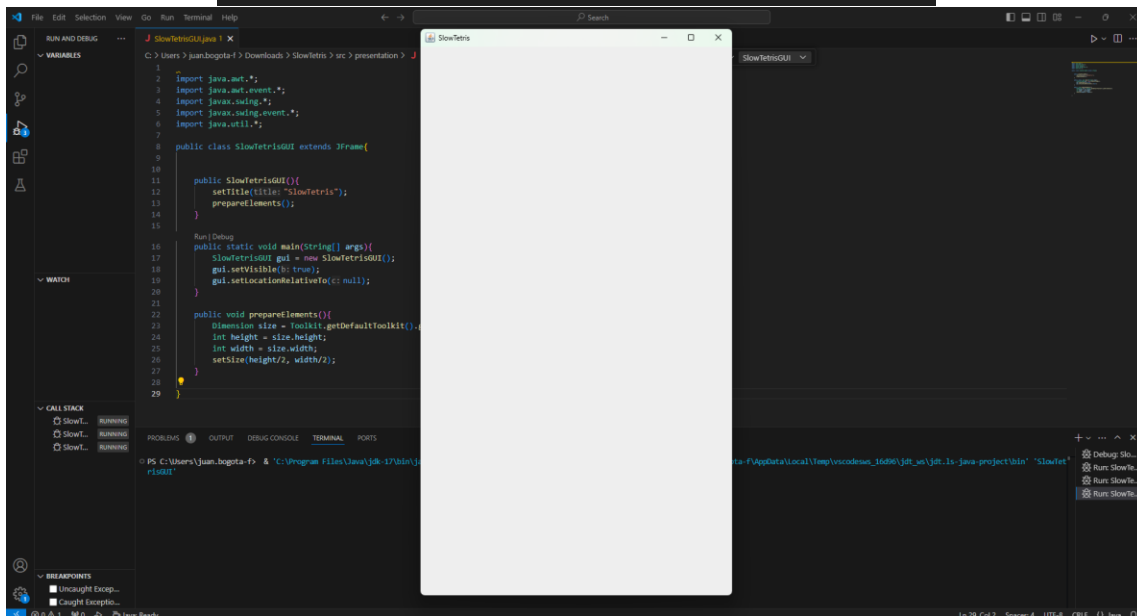
```
PS C:\Users\juan.bogota-f\Downloads\SlowTetris> java -cp bin SlowTetrisGUI
```

```
□
```



2. Modifiquen el tamaño de la ventana para que ocupe un cuarto de la pantalla y ubíquela en el centro. Para eso inicien la codificación del método `prepareElements`. Capturen esa pantalla.

```
1
2 import java.awt.*;
3 import java.awt.event.*;
4 import javax.swing.*;
5 import javax.swing.event.*;
6 import java.util.*;
7
8 public class SlowTetrisGUI extends JFrame{
9
10
11     public SlowTetrisGUI(){
12         setTitle(title: "SlowTetris");
13         prepareElements();
14     }
15
16     Run | Debug
17     public static void main(String[] args){
18         SlowTetrisGUI gui = new SlowTetrisGUI();
19         gui.setVisible(b: true);
20         gui.setLocationRelativeTo(c: null);
21     }
22
23     public void prepareElements(){
24         Dimension size = Toolkit.getDefaultToolkit().getScreenSize();
25         int height = size.height;
26         int width = size.width;
27         setSize(height/2, width/2);
28     }
29 }
```



3. Traten de cerrar la ventana. ¿Termina la ejecución? ¿Qué deben hacer en consola para terminar la ejecución?

Cierra la ventana, pero no termina la ejecución, la consola espera a nuestro comando para terminar, podemos terminar la ejecución con `ctrl+c`.

4. Estudien en `JFrame` el método `setDefaultCloseOperation`.

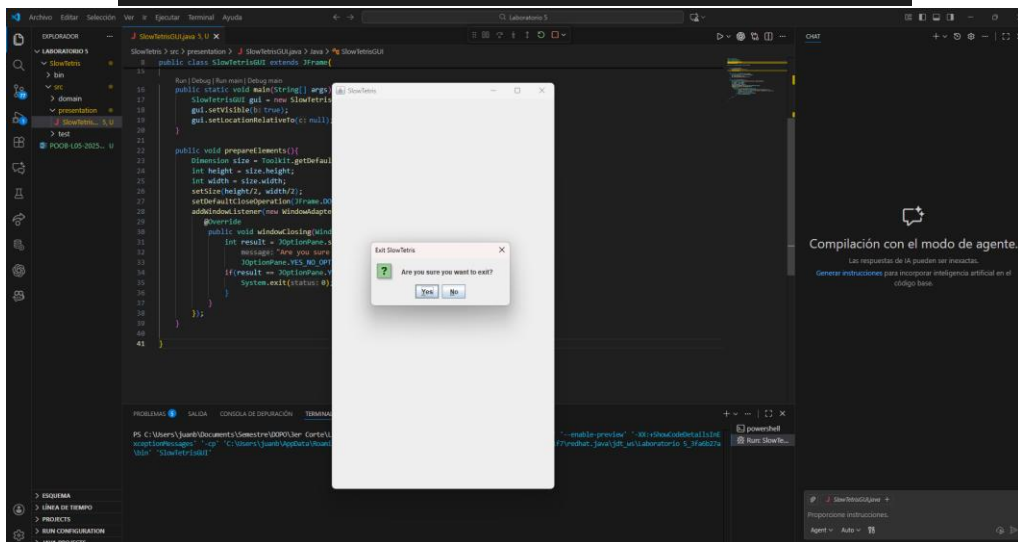
¿Para qué sirve?

- `JFrame.EXIT_ON_CLOSE`: Para salir del programa
- `JFrame.HIDE_ON_CLOSE`: Para minimizar el programa.
- `JFrame.DISPOSE_ON_CLOSE`: Elimina el marco que es el objeto principal y también se mantiene ejecutándose.
- `JFrame.DO_NOTHING_ON_CLOSE`: No hace nada ignorando el click de cierre.

¿Cómo lo usarían si queremos confirmar el cierre de la aplicación?

Para lograrlo usamos `setDefaultCloseOperation` así evitamos cerrar la ventana, con `DO_NOTHING_ON_CLOSE`. Luego con un nuevo `WindowListener` mostramos el panel para que el usuario si quiere pueda cerrar o no.

```
public void prepareElements(){
    Dimension size = Toolkit.getDefaultToolkit().getScreenSize();
    int height = size.height;
    int width = size.width;
    setSize(height/2, width/2);
    setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
    addWindowListener(new WindowAdapter(){
        @Override
        public void windowClosing(WindowEvent e){
            int result = JOptionPane.showConfirmDialog(parentComponent: null,
                message: "Are you sure you want to exit?", title: "Exit SlowTetris",
                JOptionPane.YES_NO_OPTION);
            if(result == JOptionPane.YES_OPTION){
                System.exit(status: 0);
            }
        }
    });
}
```



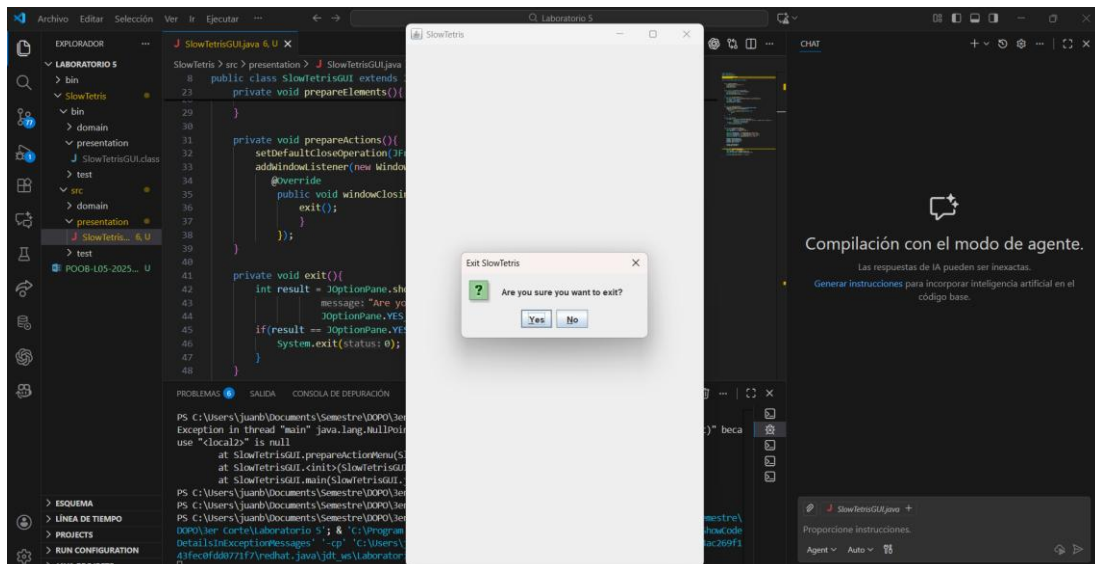
¿Cómo lo usarían si queremos simplemente cerrar la aplicación?

Simplemente tenemos que añadir `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`;

5. Preparen el “oyente” correspondiente al icono cerrar que le pida al usuario que confirme su selección. Para eso inicien la codificación del método `prepareActions` y el método asociado a la acción (`exit`). Ejecuten el programa y cierren el programa. Capturen las pantallas.

```
private void prepareActions(){
    setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
    addWindowListener(new WindowAdapter(){
        @Override
        public void windowClosing(WindowEvent e){
            exit();
        }
    });
}

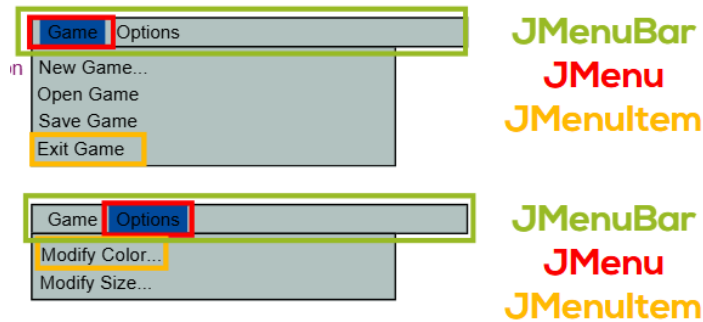
private void exit(){
    int result = JOptionPane.showConfirmDialog(this,
        message: "Are you sure you want to exit?", title: "Exit SlowTetris",
        JOptionPane.YES_NO_OPTION);
    if(result == JOptionPane.YES_OPTION){
        System.exit(status: 0);
    }
}
```



### Ciclo 1: Ventana con menú – Salir [En\*.java y lab05.doc]

El objetivo es implementar un menú clásico para la aplicación con un final adecuado desde la opción del menú para salir. El menú debe ofrecer mínimo las siguientes opciones :Nuevo, Abrir – Salvar y Salir . Incluyan los separadores de opciones.

1. Expliquen los componentes visuales necesarios para este menú. ¿Cuáles serían atributos y cuáles podrían ser variables del método `prepareElements`? Justifique.



Para un ejemplo vamos a utilizar el mockup la ventana de tetris que realizamos en la realización del trabajo en clase.

Estos componentes permiten vincular opciones de menú en nuestras ventas, tipo menú principal, como por ejemplo el conocido Inicio, Archivo, Edición etc...

**JMenuBar:**

Barra superior que contiene los menús principales de la aplicación, como Archivo, Editar, etc.

**JMenu:**

Cada menú dentro de la barra. Al hacer clic despliega una lista de opciones o submenús.

**JMenuItem:**

Opción básica dentro de un menú. Ejecuta una acción cuando se selecciona.

**JCheckBoxMenuItem:**

Elemento de menú con comportamiento de checkbox. Permite activar o desactivar una opción.

**JRadioButtonMenuItem:**

Elemento con comportamiento de radio button. Se usa para seleccionar solo una opción entre varias.

**JPopupMenu:**

Menú emergente que aparece generalmente con clic derecho. Ofrece acciones contextuales según el componente.

Considerando que el menú debe incluir, como mínimo, las opciones Nuevo, Abrir, Salvar y Salir, utilizamos un JMenu para agrupar y desplegar todas estas acciones. Cada opción individual dentro del menú se representa mediante un JMenuItem, mientras que el JMenuBar corresponde a la barra superior donde se ubica dicho menú.



|        |                   |
|--------|-------------------|
| Pig    | Martha Washington |
| Bird   | Abigail Adams     |
| Cat    | Martha Randolph   |
| Dog    | Dolley Madison    |
| Rabbit | Elizabeth Monroe  |
| Pig    | Louisa Adams      |
|        | Emily Donelson    |

JButton

## JCheckBox

## JComboBox

JList

| A Menu                                                                            | Another Menu                |
|-----------------------------------------------------------------------------------|-----------------------------|
|                                                                                   | A text-only menu item Alt+1 |
|  | Both text and icon          |
|  |                             |
| <input type="radio"/>                                                             | A radio button menu item    |
|                                                                                   | Another one                 |
|                                                                                   | A check box menu item       |
|                                                                                   | Another one                 |
|                                                                                   | A submenu ▶                 |

☐ Bird  
☐ Cat  
☐ Dog  
☐ Rabbit  
☒ Pig



[JMenu](#)

JRadioButton

JSlider

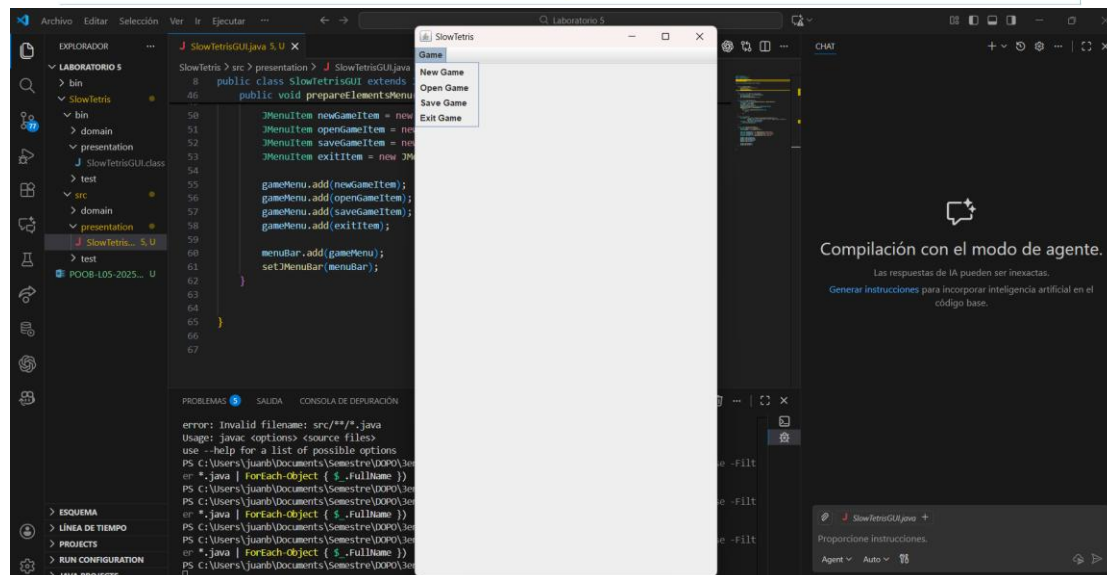
Date: 07/2006City: Santa Rosa

Enter the password:

JSpinner

UITextField

[JPasswordField](#)



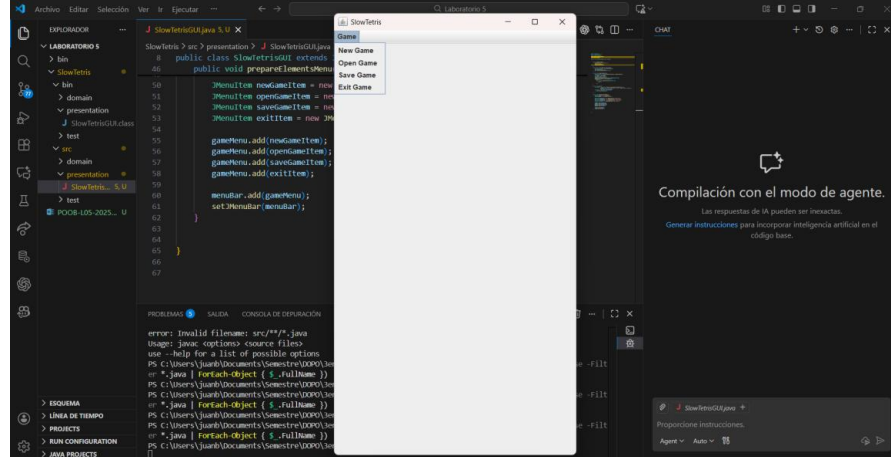
2. Construya la forma del menú propuesto (**prepareElements** - **prepareElementsMenu**) . Ejecuten. Capturen la pantalla.

```
public void prepareElementsMenu(){
    JMenuBar menuBar = new JMenuBar();
    JMenu gameMenu = new JMenu(s: "Game");

    JMenuItem newGameItem = new JMenuItem(text: "New Game");
    JMenuItem openGameItem = new JMenuItem(text: "Open Game");
    JMenuItem saveGameItem = new JMenuItem(text: "Save Game");
    JMenuItem exitItem = new JMenuItem(text: "Exit Game");

    gameMenu.add(newGameItem);
    gameMenu.add(openGameItem);
    gameMenu.add(saveGameItem);
    gameMenu.add(exitItem);

    menuBar.add(gameMenu);
    setJMenuBar(menuBar);
}
```

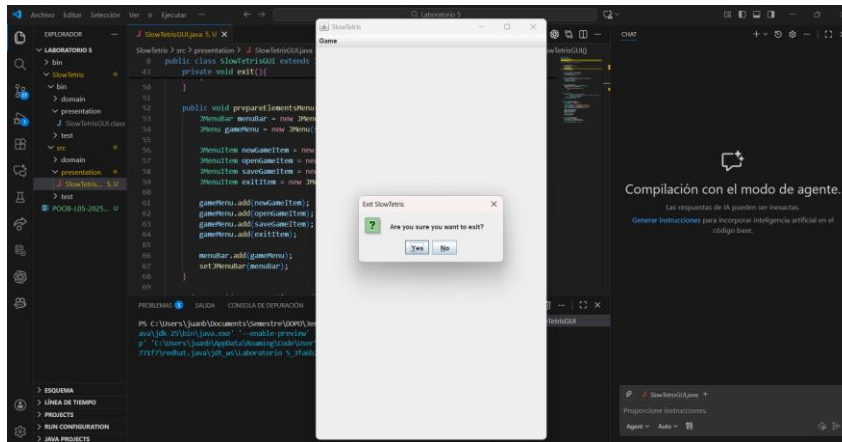
The screenshot shows an IDE with a Java file named 'SlowTetris.java'. The code implements the 'prepareElementsMenu()' method, which creates a 'JMenuBar' and a 'JMenu' named 'Game'. It then adds four 'JMenuItem' to the 'Game' menu: 'New Game', 'Open Game', 'Save Game', and 'Exit Game'. The menu is then added to the 'JMenuBar' and set on the window. A small preview window titled 'Game' is shown, displaying the menu items. The IDE interface includes a sidebar with project files, a central editor, and a bottom panel with a chat window and a console.

3. Preparen el “oyente” correspondiente al icono cerrar con confirmación (**prepareActions** - **prepareActionsMenu**). Ejecuten el programa y salgan del programa. Capturen las pantallas.

```
private void prepareActions(){
    setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
    addWindowListener(new WindowAdapter(){
        @Override
        public void windowClosing(WindowEvent e){
            exit();
        }
    });
}

private void prepareActionMenu(){
    JMenuBar menu = getJMenuBar();
    JMenu gameMenu = menu.getMenu(index: 0);
    JMenuItem exitItem = gameMenu.getItem(pos: 3);

    exitItem.addActionListener(e -> exit());
}
```



## Ciclo 2: Salvar y abrir [En\*.java y lab05.doc]

El objetivo es preparar la interfaz para las funciones de persistencia

1. Detalle el componente `JFileChooser` especialmente los métodos: `JFileChooser`, `showOpenDialog`, `showSaveDialog`, `getSelectedFile`.

**JFileChooser:** Es una clase java que nos permite mostrar fácilmente una ventana para la selección de un fichero, presentara los siguientes valores:

- **JFileChooser.CANCEL\_OPTION:** Si el usuario le ha dado al botón de cancelar.
- **JFileChooser.APPROVE\_OPTION:** Si el usuario le ha dado al botón de aceptar.
- **JFileChooser.ERROR\_OPTION:** Si ha ocurrido algún error.

Ejemplo de uso general de JFileChooser:

```

1  import javax.swing.*;
2  import java.io.File;
3
4  public class EjemploJFileChooser {
5
6      public static void main(String[] args) {
7
8          // Crear el JFileChooser
9          JFileChooser fileChooser = new JFileChooser();
10
11         // Mostrar el cuadro de diálogo de selección
12         int resultado = fileChooser.showOpenDialog(null);
13
14         // Verificar la opción seleccionada
15         if (resultado == JFileChooser.APPROVE_OPTION) {
16             File archivo = fileChooser.getSelectedFile();
17             System.out.println("Archivo seleccionado: " + archivo.getAbsolutePath());
18
19         } else if (resultado == JFileChooser.CANCEL_OPTION) {
20             System.out.println("El usuario canceló la operación.");
21
22         } else if (resultado == JFileChooser.ERROR_OPTION) {
23             System.out.println("Ocurrió un error al intentar seleccionar un archivo.");
24         }
25     }
26 }
27
  
```

showOpenDialog:

Para abrir un archivo y poder leer su contenido, se utiliza el método showOpenDialog().

Este método despliega una ventana del tipo JFileChooser, permitiendo al usuario seleccionar el fichero que desea abrir.

Ejemplo de showOpenDialog() – Abrir un archivo:

```
1  JFileChooser fileChooser = new JFileChooser();
2  int opcion = fileChooser.showOpenDialog(null);
3
4  if (opcion == JFileChooser.APPROVE_OPTION) {
5      File archivo = fileChooser.getSelectedFile();
6      System.out.println("Archivo seleccionado para abrir: " + archivo.getAbsolutePath());
7  }
```

showSaveDialog:

Si lo que necesitamos es elegir un archivo para guardar información, el proceso es similar, pero se usa el método showSaveDialog(). Una vez que el usuario confirma la selección, podemos obtener el archivo elegido mediante getSelectedFile() y proceder a escribir en él los datos que necesitemos guardar.

Ejemplo de showSaveDialog() – Guardar un archivo:

```
1  JFileChooser fileChooser = new JFileChooser();
2  int opcion = fileChooser.showSaveDialog(null);
3
4  if (opcion == JFileChooser.APPROVE_OPTION) {
5      File archivo = fileChooser.getSelectedFile();
6      System.out.println("Archivo seleccionado para guardar: " + archivo.getAbsolutePath());
7      // Aquí podrías escribir datos en el archivo.
8  }
```

getSelectedFile:

Tal como indica su nombre, este método permite obtener el archivo que el usuario seleccionó en el cuadro de diálogo del JFileChooser, ya sea para abrirlo o para guardarlo.

Ejemplo usando getSelectedFile() directamente después del diálogo:

```
1  JFileChooser fileChooser = new JFileChooser();
2  int opcion = fileChooser.showOpenDialog(null);
3
4  if (opcion == JFileChooser.APPROVE_OPTION) {
5      File archivoSeleccionado = fileChooser.getSelectedFile();
6      System.out.println("Se obtuvo el archivo: " + archivoSeleccionado.getName());
7  }
```

- Implementen parcialmente los elementos necesarios para salvar y abrir. Al seleccionar los archivos indiquen que las funcionalidades están en construcción detallando la acción y el nombre del archivo seleccionado.

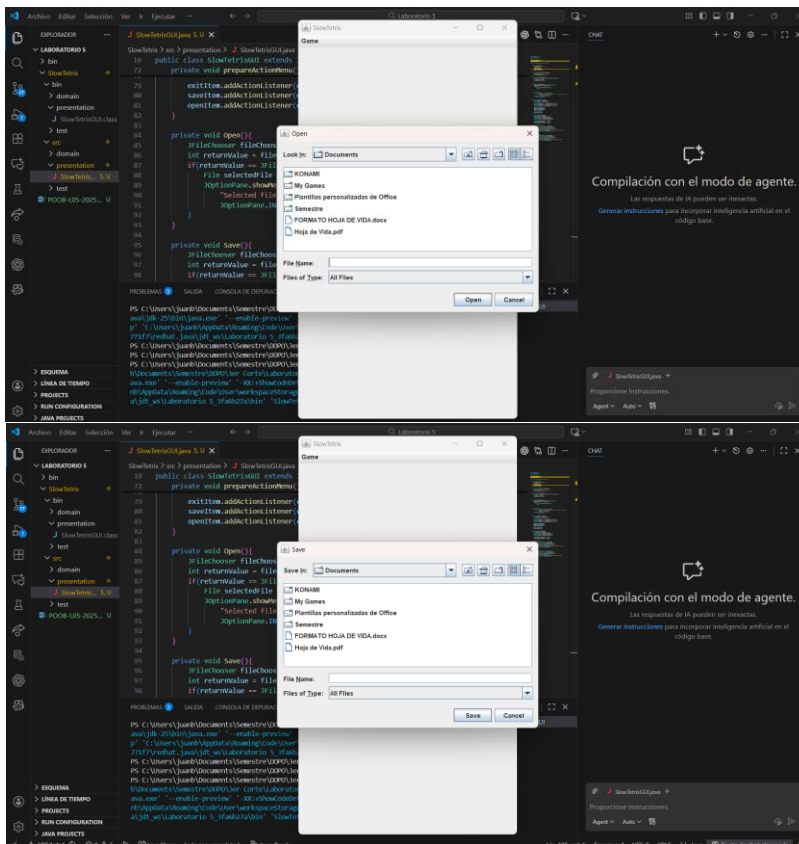
```
private void prepareActionMenu(){
    JMenuBar menu = getJMenuBar();
    JMenu gameMenu = menu.getMenu(index: 0);
    JMenuItem exitItem = gameMenu.getItem(pos: 3);
    JMenuItem saveItem = gameMenu.getItem(pos: 2);
    JMenuItem openItem = gameMenu.getItem(pos: 1);

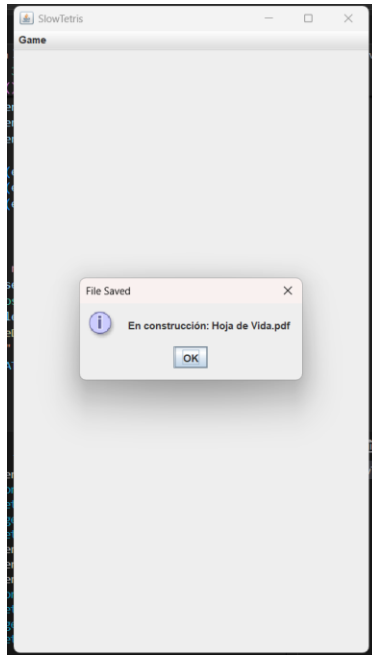
    exitItem.addActionListener(e -> exit());
    saveItem.addActionListener(e -> Save());
    openItem.addActionListener(e -> Open());
}

private void Open(){
    JFileChooser fileChooser = new JFileChooser();
    int returnValue = fileChooser.showOpenDialog(this);
    if(returnValue == JFileChooser.APPROVE_OPTION){
        File selectedFile = fileChooser.getSelectedFile();
        JOptionPane.showMessageDialog(this,
            "En construcción: " + selectedFile.getName(), title: "File Opened",
            JOptionPane.INFORMATION_MESSAGE);
    }
}

private void Save(){
    JFileChooser fileChooser = new JFileChooser();
    int returnValue = fileChooser.showSaveDialog(this);
    if(returnValue == JFileChooser.APPROVE_OPTION){
        File selectedFile = fileChooser.getSelectedFile();
        JOptionPane.showMessageDialog(this,
            "En construcción: " + selectedFile.getName(), title: "File Saved",
            JOptionPane.INFORMATION_MESSAGE);
    }
}
```

- Ejecuten las dos opciones y capturen las pantallas más significativas.

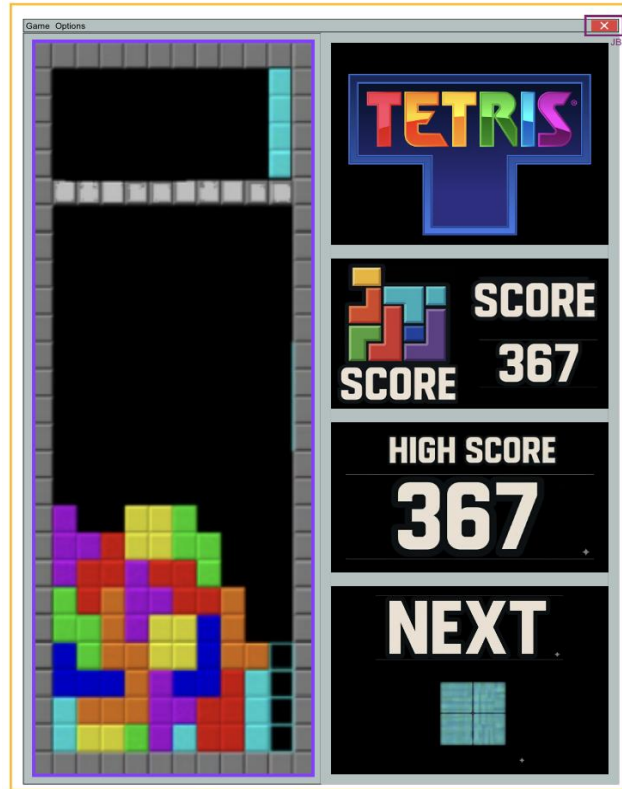




### Ciclo 3: Forma de la ventana principal [En \*.java y lab05.doc]

El objetivo es codificar el diseño de la ventana principal (todos los elementos de primer nivel)

1. Presenten el bosquejo del diseño de interfaz con todos los componentes necesarios. (Incluyan la imagen)



2. Continúen con la implementación definiendo los atributos necesarios y extendiendo el método `prepareElements()`.

Para la zona del tablero definan un método `prepareElementsBoard()` y un método `refresh()` que actualiza la vista del tablero considerando, por ahora, el tablero inicial por omisión. Este método lo vamos a implementar realmente en otros ciclos.

```
private void prepareElementsInfo() {  
    infoPanel = new JPanel();  
    infoPanel.setLayout(new GridLayout(4, 1, 10, 10));  
    infoPanel.setBackground(background-color);  
    infoPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));  
  
    prepareTitlePanel();  
    prepareNextPiecePanel();  
    prepareScorePanel();  
    prepareHighScorePanel();  
  
    infoPanel.add(titlePanel);  
    infoPanel.add(nextPiecePanel);  
    infoPanel.add(scorePanel);  
    infoPanel.add(highScorePanel);  
  
    add(infoPanel, BorderLayout.CENTER);  
}
```

```

private void prepareTitlePanel() {
    titlePanel = new JPanel() {
        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            drawTetrisTitle(g);
        }
    };
    titlePanel.setPreferredSize(new Dimension(300, 120));
    titlePanel.setBackground(Color.BLACK);
    titlePanel.setBorder(BorderFactory.createLineBorder(new Color(100, 100, 255), 3));
}

```

```

private void drawTetrisTitle(Graphics g) {
    Graphics2D g2d = (Graphics2D) g;
    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

    String[] letters = {"T", "E", "T", "R", "I", "S"};
    Color[] colors = {Color.RED, Color.GREEN, Color.BLUE, Color.YELLOW, Color.MAGENTA, Color.CYAN};

    Font font = new Font("Arial", Font.BOLD, 40);
    g2d.setFont(font);

    int x = 20;
    int y = 60;

    for (int i = 0; i < letters.length; i++) {
        g2d.setColor(colors[i]);
        g2d.drawString(letters[i], x, y);
        x += 45;
    }

    int pieceX = 180;
    int pieceY = 70;
    int blockSize = 20;

    g2d.setColor(Color.MAGENTA);
    g2d.fillRect(pieceX, pieceY, blockSize, blockSize);
    g2d.fillRect(pieceX + blockSize, pieceY, blockSize, blockSize);
    g2d.fillRect(pieceX + blockSize * 2, pieceY, blockSize, blockSize);
    g2d.fillRect(pieceX + blockSize, pieceY + blockSize, blockSize, blockSize);

    g2d.setColor(Color.BLACK);
    g2d.drawRect(pieceX, pieceY, blockSize, blockSize);
    g2d.drawRect(pieceX + blockSize, pieceY, blockSize, blockSize);
    g2d.drawRect(pieceX + blockSize * 2, pieceY, blockSize, blockSize);
    g2d.drawRect(pieceX + blockSize, pieceY + blockSize, blockSize, blockSize);
}

```



```

private void prepareNextPiecePanel() {
    nextPiecePanel = new JPanel() {
        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            drawNextPiece(g);
        }
    };
    nextPiecePanel.setBackground(Color.BLACK);
    nextPiecePanel.setBorder(BorderFactory.createLineBorder(gridColor, 2));
    nextPiecePanel.setPreferredSize(new Dimension(300, 120));
}

private void drawNextPiece(Graphics g) {
    Graphics2D g2d = (Graphics2D) g;
    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

    g2d.setColor(Color.WHITE);
    g2d.setFont(new Font("Arial", Font.BOLD, 30));
    g2d.drawString("NEXT", 20, 40);

    int blockSize = 20;
    int startX = 180;
    int startY = 50;

    g2d.setColor(Color.CYAN);
    g2d.fillRect(startX, startY, blockSize, blockSize);
    g2d.fillRect(startX + blockSize, startY, blockSize, blockSize);
    g2d.fillRect(startX, startY + blockSize, blockSize, blockSize);
    g2d.fillRect(startX + blockSize, startY + blockSize, blockSize, blockSize);

    g2d.setColor(Color.WHITE);
    g2d.drawRect(startX, startY, blockSize, blockSize);
    g2d.drawRect(startX + blockSize, startY, blockSize, blockSize);
    g2d.drawRect(startX, startY + blockSize, blockSize, blockSize);
    g2d.drawRect(startX + blockSize, startY + blockSize, blockSize, blockSize);
}

```

```

private void prepareScorePanel() {
    scorePanel = new JPanel();
    scorePanel.setLayout(new BorderLayout());
    scorePanel.setBackground(Color.BLACK);
    scorePanel.setBorder(BorderFactory.createLineBorder(gridColor, 2));

    JPanel iconPanel = new JPanel() {
        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            drawScoreIcon(g);
        }
    };
    iconPanel.setPreferredSize(new Dimension(100, 100));
    iconPanel.setBackground(Color.BLACK);

    JPanel textPanel = new JPanel(new GridLayout(2, 1));
    textPanel.setBackground(Color.BLACK);

    JLabel scoreTitle = new JLabel("SCORE");
    scoreTitle.setForeground(Color.WHITE);
    scoreTitle.setFont(new Font("Arial", Font.BOLD, 25));
    scoreTitle.setHorizontalAlignment(SwingConstants.CENTER);

    scoreLabel = new JLabel("367");
    scoreLabel.setForeground(Color.WHITE);
    scoreLabel.setFont(new Font("Arial", Font.BOLD, 50));
    scoreLabel.setHorizontalAlignment(SwingConstants.CENTER);

    textPanel.add(scoreTitle);
    textPanel.add(scoreLabel);

    scorePanel.add(iconPanel, BorderLayout.WEST);
    scorePanel.add(textPanel, BorderLayout.CENTER);
}

```

```

private void drawScoreIcon(Graphics g) {
    int blockSize = 15;
    Color[] colors = {Color.ORANGE, Color.CYAN, Color.GREEN, Color.RED, Color.MAGENTA, Color.YELLOW};

    int[][] positions = {
        {10, 20}, {25, 20}, {10, 35}, {25, 35},
        {40, 35}, {55, 20}, {40, 50}, {55, 35}
    };

    for (int i = 0; i < positions.length && i < colors.length; i++) {
        g.setColor(colors[i % colors.length]);
        g.fillRect(positions[i][0], positions[i][1], blockSize, blockSize);
        g.setColor(Color.BLACK);
        g.drawRect(positions[i][0], positions[i][1], blockSize, blockSize);
    }
}

private void prepareHighScorePanel() {
    highScorePanel = new JPanel(new GridLayout(2, 1));
    highScorePanel.setBackground(Color.BLACK);
    highScorePanel.setBorder(BorderFactory.createLineBorder(gridColor, 2));

    JLabel highScoreTitle = new JLabel("HIGH SCORE");
    highScoreTitle.setForeground(Color.WHITE);
    highScoreTitle.setFont(new Font("Arial", Font.BOLD, 25));
    highScoreTitle.setHorizontalAlignment(SwingConstants.CENTER);

    highScoreLabel = new JLabel("367");
    highScoreLabel.setForeground(Color.WHITE);
    highScoreLabel.setFont(new Font("Arial", Font.BOLD, 50));
    highScoreLabel.setHorizontalAlignment(SwingConstants.CENTER);

    highScorePanel.add(highScoreTitle);
    highScorePanel.add(highScoreLabel);
}

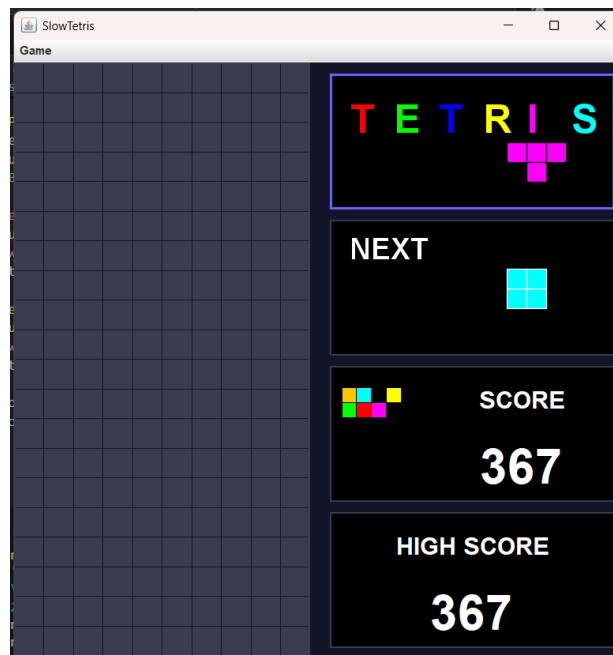
```

```

public void refresh() {
    if (boardPanel != null) {
        boardPanel.repaint();
    }
    if (nextPiecePanel != null) {
        nextPiecePanel.repaint();
    }
}
}

```

3. Ejecuten y capturen la pantalla.



## Ciclo 4: Cambiar colores [En \*.java y lab05.doc]

El objetivo es implementar este caso de uso.

1. **Expliquen los elementos (vista – controlador) necesarios para implementar este caso de uso.**

Para implementar la funcionalidad de cambiar colores en Tetris, es necesario utilizar los siguientes elementos de la vista y del controlador:

- **JMenuItem “Modify Colors” (vista):**  
En el menú Options ya existe un ítem llamado Modify Colors, el cual servirá como disparador de la acción. Cuando el usuario seleccione esta opción, se ejecutará el comportamiento encargado de modificar los colores del juego.
- **Controlador mediante ActionListener (controlador):**  
Para registrar la acción generada por el usuario al seleccionar *Modify Colors*, es necesario agregar un ActionListener a este JMenuItem. Este listener será el encargado de ejecutar el cambio de colores en los paneles del juego.
- **Actualización de colores en los paneles relevantes (vista):**  
Los elementos que deben actualizar su color son principalmente:
  - El panel del tablero (boardPanel) que actualmente usa backgroundColor y gridColor.
  - Los paneles de información como titlePanel, nextPiecePanel, scorePanel, y highScorePanel, los cuales usan colores fijos que pueden reemplazarse por variantes dinámicas.

Cambiar estos colores implica actualizar los atributos backgroundColor y gridColor o establecer nuevos valores de color según se requiera.

- **Método refresh() para repintar (vista/controlador):**  
Una vez hechos los cambios, es necesario llamar al método refresh(), el cual fuerza el repintado del tablero y del panel de la siguiente pieza, garantizando que los nuevos colores se vean inmediatamente.

2. **Detalle el comportamiento de JColorChooser especialmente el método estático ShowDialog:**

JColorChooser es una clase que permite abrir un selector visual de colores. En tu código, el método estático showDialog muestra una ventana modal donde el usuario puede elegir un color para una pieza específica. El diálogo se abre con un título personalizado y con currentColor como color inicial.

El showDialog devuelve el color seleccionado si el usuario confirma la operación, o null si cancela. Por eso, tras obtener el resultado, el programa valida el color y, si es válido, actualiza el color de la pieza, cambia la vista previa y refresca la interfaz. El showDialog gestiona toda la interacción de selección de color y entrega el valor final para aplicarlo en la interfaz del juego.

### 3. Implementen los componentes necesarios para cambiar el color de las piezas.

```
private void prepareModifyColors(){
    JDialog dialog = new JDialog(this, title: "Modify Piece Colors", modal: true);
    dialog.setLayout(new BorderLayout(hgap: 10, vgap: 10));
    dialog.setSize(width: 400, height: 500);
    dialog.setLocationRelativeTo(this);

    JPanel mainPanel = new JPanel();
    mainPanel.setLayout(new GridLayout(rows: 7, cols: 1, hgap: 10, vgap: 10));
    mainPanel.setBorder(BorderFactory.createEmptyBorder(top: 20, left: 20, bottom: 20, right: 20));

    JLabel instructionLabel = new JLabel(text: "Select a piece to change its color:");
    instructionLabel.setHorizontalAlignment(SwingConstants.CENTER);
    instructionLabel.setFont(new Font(name: "Arial", Font.BOLD, size: 14));

    dialog.add(instructionLabel, BorderLayout.NORTH);

    createPieceColorButton(mainPanel, text: "I - Line Piece", colorI, pieceType: "I");
    createPieceColorButton(mainPanel, text: "O - Square Piece", colorO, pieceType: "O");
    createPieceColorButton(mainPanel, text: "T - T Piece", colorT, pieceType: "T");
    createPieceColorButton(mainPanel, text: "S - S Piece", colorS, pieceType: "S");
    createPieceColorButton(mainPanel, text: "Z - Z Piece", colorZ, pieceType: "Z");
    createPieceColorButton(mainPanel, text: "J - J Piece", colorJ, pieceType: "J");
    createPieceColorButton(mainPanel, text: "L - L Piece", colorL, pieceType: "L");

    dialog.add(mainPanel, BorderLayout.CENTER);

    JButton closeButton = new JButton(text: "Close");
    closeButton.addActionListener(e -> dialog.dispose());
    JPanel buttonPanel = new JPanel();
    buttonPanel.add(closeButton);
    dialog.add(buttonPanel, BorderLayout.SOUTH);

    dialog.setVisible(b: true);
}
```

```
private void createPieceColorButton(JPanel panel, String text, Color currentColor, String pieceType){
    JPanel piecePanel = new JPanel(new BorderLayout(hgap: 10, vgap: 0));

    JLabel label = new JLabel(text);
    label.setFont(new Font(name: "Arial", Font.PLAIN, size: 14));

    JPanel colorPreview = new JPanel();
    colorPreview.setBackground(currentColor);
    colorPreview.setPreferredSize(new Dimension(width: 50, height: 30));
    colorPreview.setBorder(BorderFactory.createLineBorder(Color.BLACK, thickness: 2));

    JButton changeButton = new JButton(text: "Change Color");
    changeButton.addActionListener(e -> {
        Color newColor = JColorChooser.showDialog(this,
            "Choose color for " + text, currentColor);
        if(newColor != null){
            updatePieceColor(pieceType, newColor);
            colorPreview.setBackground(newColor);
            refresh();
        }
    });

    piecePanel.add(label, BorderLayout.WEST);
    piecePanel.add(colorPreview, BorderLayout.CENTER);
    piecePanel.add(changeButton, BorderLayout.EAST);

    panel.add(piecePanel);
}
```

```
private void updatePieceColor(String pieceType, Color newColor){
    switch(pieceType){
        case "I" -> colorI = newColor;
        case "O" -> colorO = newColor;
        case "T" -> colorT = newColor;
        case "S" -> colorS = newColor;
        case "Z" -> colorZ = newColor;
        case "J" -> colorJ = newColor;
        case "L" -> colorL = newColor;
    }
}
```

```

public class SlowTetrisGUI extends JFrame{

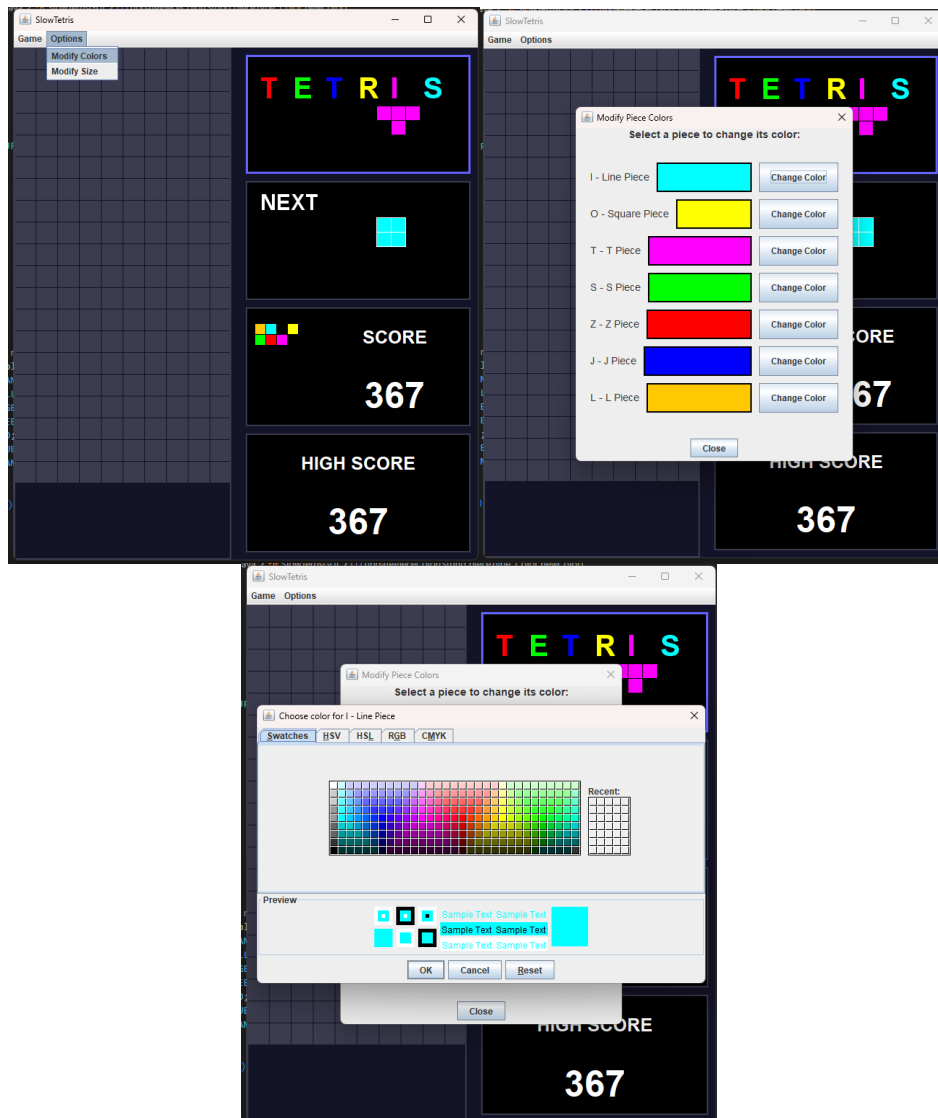
    //paneles
    private JPanel boardPanel;
    private JPanel infoPanel;
    private JPanel titlePanel;
    private JPanel nextPiecePanel;
    private JPanel scorePanel;
    private JPanel highScorePanel;

    //etiquetas
    private JLabel scoreLabel;
    private JLabel highScoreLabel;

    //colores
    private Color backgroundColor = new Color(r: 20, g: 20, b: 40);
    private Color gridColor = new Color(r: 60, g: 60, b: 80);
    private Color colorI = Color.CYAN; // Pieza I (línea)
    private Color colorO = Color.YELLOW; // Pieza O (cuadrado)
    private Color colorT = Color.MAGENTA; // Pieza T
    private Color colorS = Color.GREEN; // Pieza S
    private Color colorZ = Color.RED; // Pieza Z
    private Color colorJ = Color.BLUE; // Pieza J
    private Color colorL = Color.ORANGE; // Pieza L

```

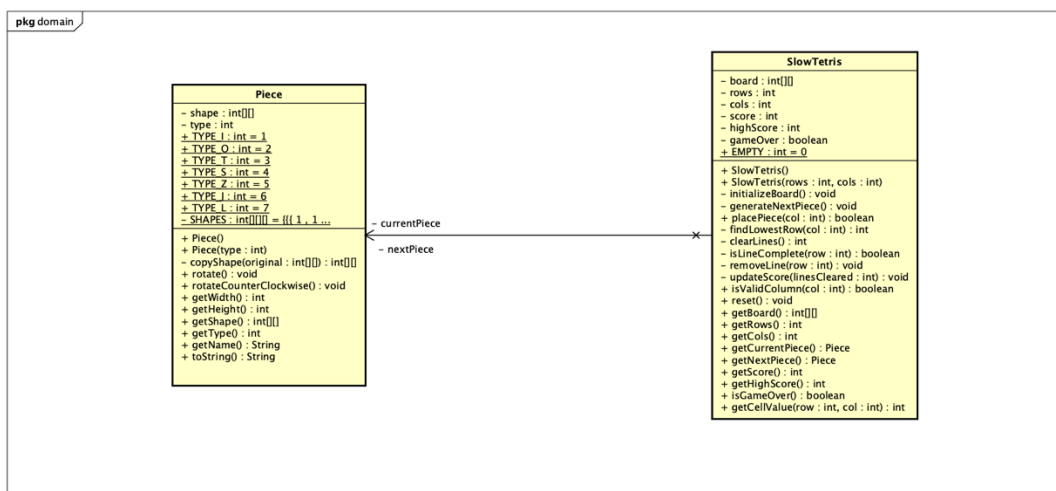
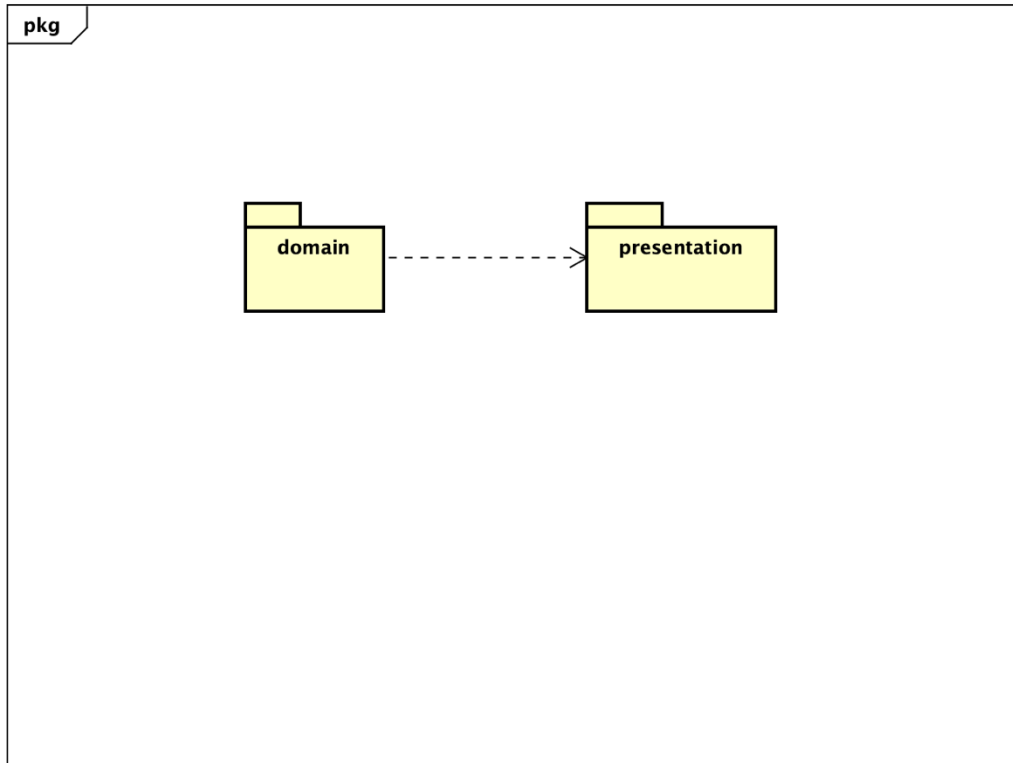
4. Ejecuten el caso de uso y capture las pantallas más significativas.

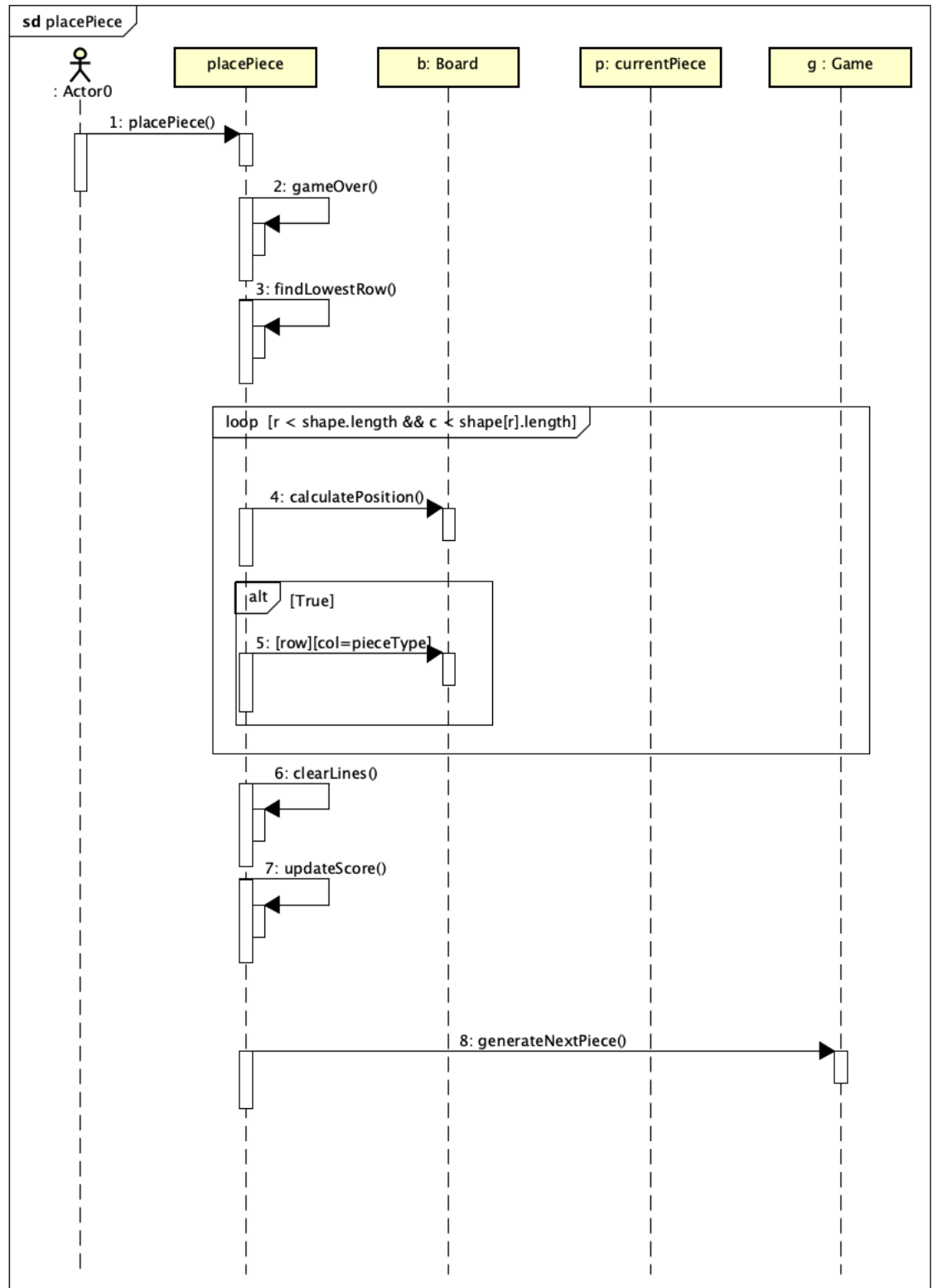


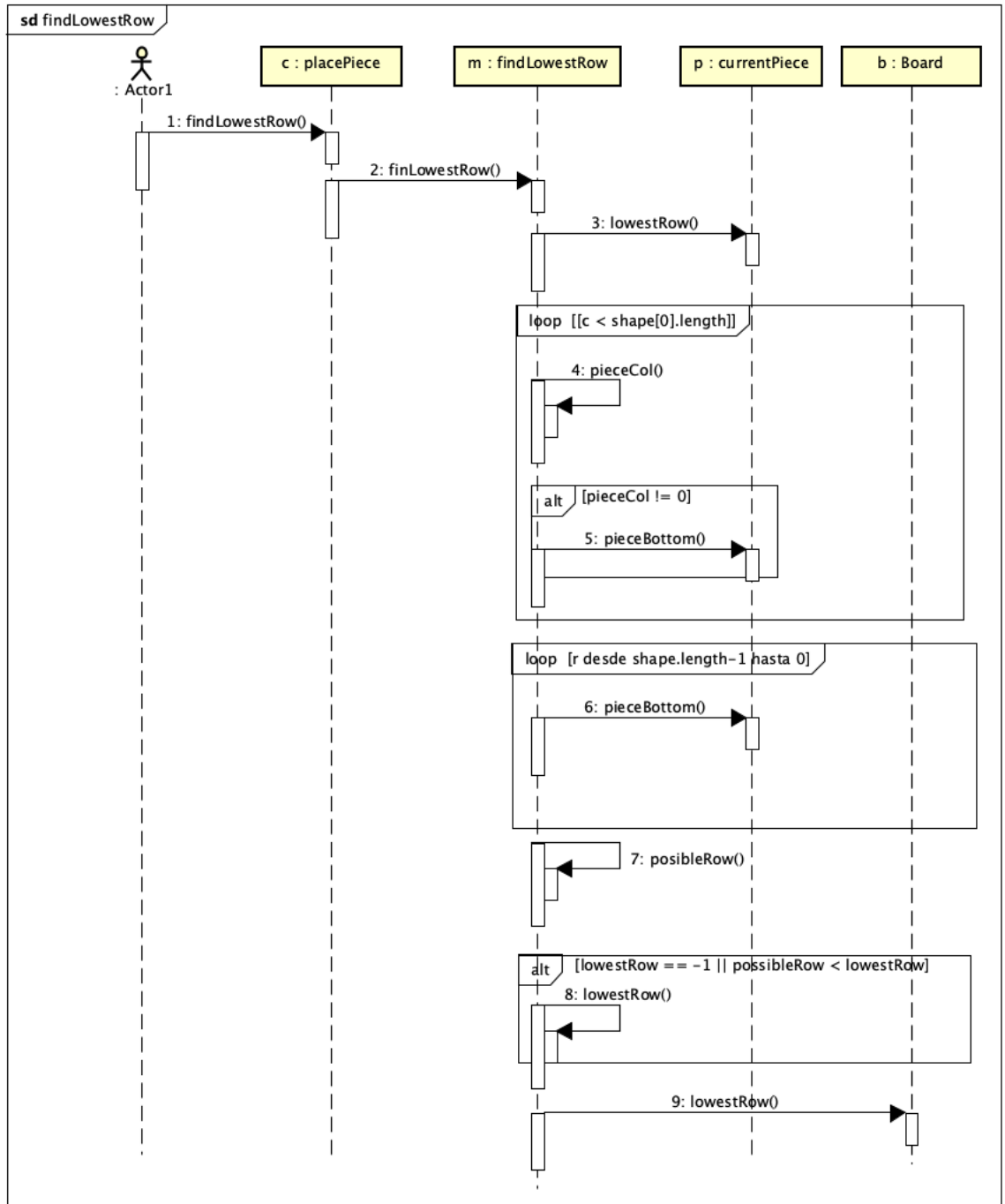
## Ciclo 5: Modelo SlowTetris [En \*.java y lab05.doc]

El objetivo es implementar la capa de dominio para SlowTetris

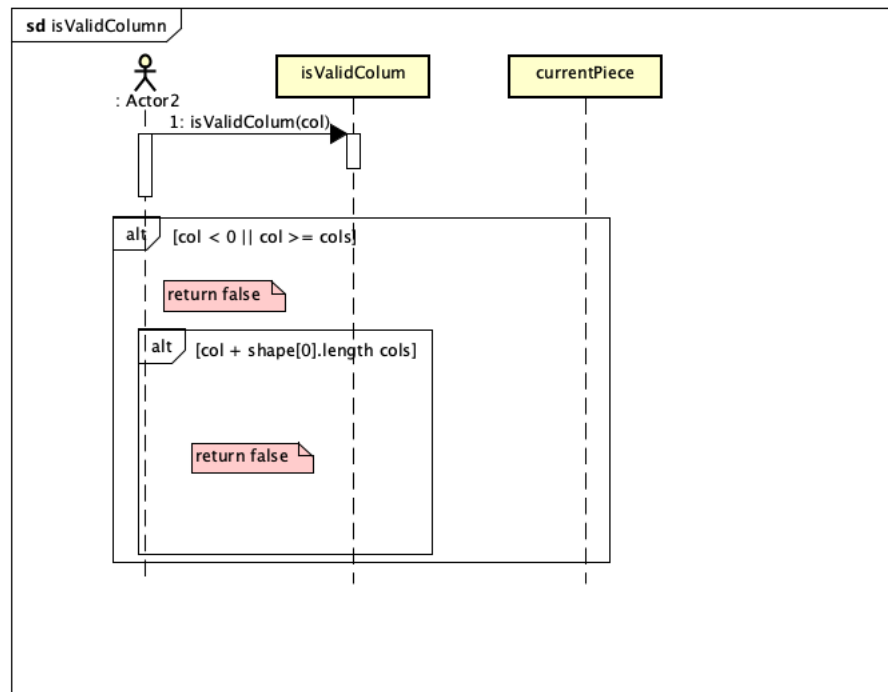
1. Construya los métodos básicos del juego (No olvide MDD y BDD)



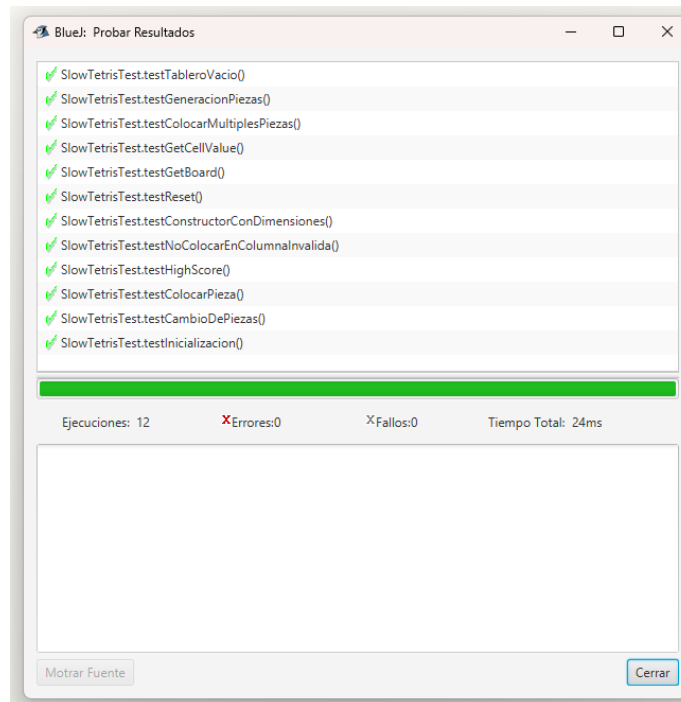








2. Ejecuten las pruebas y capturen el resultado.



### Ciclo 6: Jugar: Preparar la pieza [En \*.java y lab05.doc]

El objetivo es implementar el caso de uso jugar: preparar la pieza.

1. Adicione a la capa de presentación el atributo correspondiente al modelo.

```
src > presentation > J SlowTetrisGUI.java > J
1  package presentation;
2
3  import domain.*;
4  import java.awt.*;

private SlowTetris game;
```

2. Perfeccionen el método `refresh()` considerando la información del modelo de dominio.

```
public void refresh() {

    scoreLabel.setText(String.valueOf(game.getScore()));
    highScoreLabel.setText(String.valueOf(game.getHighScore()));

    boardPanel.repaint();
    nextPiecePanel.repaint();
    titlePanel.repaint();
    scorePanel.repaint();

    setTitle("SlowTetris - Score: " + game.getScore());
}
```

3. Expliquen los elementos necesarios para implementar este caso de uso.  
[La pieza aparece aleatoriamente. Es posible rotarla y desplazarla. Queda lista para caer]

Los elementos necesarios para implementar este caso serian:

- Generación aleatoria de piezas
- Rotación de piezas
- Obtención de pieza actual
- Eventos de teclado para rotar
- Eventos de teclado para mover
- Evento para hacer caer
- Atributo `pieceColumn` en `SlowTetrisGUI`
- Panel `previewTopPanel` para mostrar la pieza
- Método `prepareTopPreviewPanel()`
- Método `drawTopPreview()`
- Modificar `prepareElements()` para integrar el panel superior
- Actualizar `pieceColumn` en los eventos de teclado
- Resetear `pieceColumn` después de colocar pieza

4. Implementen los componentes necesarios para jugar .¿Cuántos oyentes necesitan?  
¿Por qué?

Se necesitan 2 oyentes, WindowListener para cerrar la ventana, KeyListener para mover las fichas y soltar la ficha.

```
private void prepareActions(){
    setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
    addWindowListener(new WindowAdapter(){
        @Override
        public void windowClosing(WindowEvent e){
            exit();
        }
    });

    addKeyListener(new KeyAdapter() {
        @Override
        public void keyPressed(KeyEvent e) {
            handleKeyPress(e);
        }
    });

    setFocusable(focusable: true);
    requestFocus();
}
```

```
private void handleKeyPress(KeyEvent e) {
    switch (e.getKeyCode()) {
        case KeyEvent.VK_LEFT -> {
            if (pieceColumn > 0) {
                pieceColumn--;
                previewTopPanel.repaint();
            }
        }

        case KeyEvent.VK_RIGHT -> {
            Piece currentPiece = game.getCurrentPiece();
            if (currentPiece != null && pieceColumn + currentPiece.getWidth() < game.getCols()) {
                pieceColumn++;
                previewTopPanel.repaint();
            }
        }

        case KeyEvent.VK_SPACE, KeyEvent.VK_ENTER -> {
            if (!game.isGameOver() && game.isValidColumn(pieceColumn)) {
                game.placePiece(pieceColumn);
                pieceColumn = 4;
                updateGameDisplay();
            }
        }

        case KeyEvent.VK_R -> {
            if (game.getCurrentPiece() != null) {
                game.getCurrentPiece().rotate();

                Piece piece = game.getCurrentPiece();
                if (pieceColumn + piece.getWidth() > game.getCols()) {
                    pieceColumn = game.getCols() - piece.getWidth();
                }

                previewTopPanel.repaint();
            }
        }

        case KeyEvent.VK_N -> newGame();
    }
}
```

## RETROSPECTIVA

1. **¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas/Hombre)**

Juan Daniel Bogotá Fuentes 38 horas, más o menos 5 horas por día (jueves, lunes, martes, miércoles, jueves, viernes, sábado).

Nicolas Felipe Bernal Gallo 38 horas (jueves[5 horas], lunes[5 horas], martes[5 horas], miércoles[5 hora], jueves[5 horas], viernes[5 horas], sábado[5 horas])

2. **¿Cuál es el estado actual del laboratorio? ¿Por qué?**

El estado actual del laboratorio es completo. Porque pudimos realizar en su totalidad el laboratorio, profundizando en cada uno de los puntos y sus requerimientos.

3. **Seleccionen una práctica XP del laboratorio ¿por qué consideran que es importante?**

Dos programadores trabajan juntos en el código (Pair Programming). Esta es la práctica XP que nos fue más útil a la hora de realizar este laboratorio, puesto que nosotros nos intercambiábamos la labor de escribir código y revisar el código de la otra persona. También porque intercambiamos ideas y conocimiento que fuimos adquiriendo en el auto estudio de cada persona.

4. **¿Cuál consideran fue el mayor logro? ¿Por qué? ¿Cuál consideran que fue el mayor problema? ¿Qué hicieron para resolverlo?**

Entender todos los requerimientos que exigía el laboratorio a partir de las preguntas que fueron expuestas para su resolución. No dejando por nuestra parte lugar a la ambigüedad de nuestras respuestas.

5. **¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?**

Repartir de una buena forma el trabajo para que ambos aprendamos y además tengamos la misma carga de trabajo. Nos comprometemos a repartir mejor las horas por día para hacer el laboratorio.

6. **¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.**

ChatGPT. (s/f). Chatgpt.com. Recuperado el 15 de noviembre de 2025, de <https://chatgpt.com/>  
Rodríguez, A. (s. f.). Análisis y casos. Grafos - software para la construcción, edición y análisis de grafos. Recuperado el 15 de noviembre de 2025, de <https://arodrigu.webs.upv.es/grafos/doku.php?id=analisis>

Oracle. (2024). Interfaces (The Java™ Tutorials: Learning the Java Language > Interfaces and Inheritance).  
<https://docs.oracle.com/javase/tutorial/java/landl/createinterface.html>